Urban Expansion and Green Space Loss in Thiruvananthapuram, 2000–2025

Spatial Big Data – Course Project Report

Submitted by	Sebin George	**Submitted to**	Manoj T K
Roll No.	253309	**Date**	November 2026
Programme	MSc, Semester 3	**Institution**	Digital University Kerala

Abstract. In this project I looked at how much green cover Thiruvananthapuram Corporation (101 wards, 212.7 km²) has lost to building between 2000 and 2025, and which wards lost the most. I made dry-season Landsat and Sentinel-2 composites in Google Earth Engine, classified them with Random Forest, and did the heavy spatial joins in Apache Sedona and PostGIS. Built-up land went from 15.6% to 22.9% of the corporation and green cover went from 77.7% to 74.2%. In total, 12.2 km² of green land became built-up. Most of this happened in the southern and central wards, close to roads and to areas that were already built up, and these places also got hotter. The loss has slowed down over time, from 0.71 km² a year in 2000–2010 to 0.52 km² a year in 2017–2025.

Keywords: urban expansion, green space, Landsat, Sentinel-2, Random Forest, Apache Sedona, PostGIS, spatial partitioning, association rules, hotspot analysis

1. Introduction

Anyone who has travelled around Thiruvananthapuram knows it is a very green city. But most of this green is not forest or parks. It is homestead land: houses with coconut palms, banana, jackfruit trees and small paddy patches around them. When a new house is built, usually only part of the garden is cleared, and the trees that are left can hide the roof in a satellite image. So the growth of the city is not always easy to see from above.

Over the last 25 years the city has grown along the NH 66 bypass, around Technopark in the north and towards the Vizhinjam port in the south. I wanted to find out how much green cover was lost because of this, and in which wards. This matters for three practical reasons. Green areas keep the city cooler, low-lying paddy fields and wetlands hold monsoon water, and green area per person is a simple way to measure how liveable a ward is. Ward councillors and the planning office need these numbers ward by ward, not as one figure for the whole city.

It is also a big data problem, which is why it suits this course. One Sentinel-2 composite of the study area has about 7 million 10 m pixels per band, and the analysis has to join millions of pixels with ward boundaries, roads and around 203,000 building footprints. So the project had two goals:

1. Measure how much green land turned into built-up land in the 101 wards for 2000, 2010, 2017 and 2025, find where it is clustered and what makes it more likely, and rank the wards that need attention first.
2. Do the heavy spatial processing with the tools from the course (Spark with Apache Sedona, PostGIS and FP-Growth in Spark MLlib) and check what each tool actually adds.

2. Data and Methods

2.1 Study area and base data

I used the 2024 K-SMART ward boundaries for the corporation (local body code C010100, 101 wards). The Census 2011 wards do not match these new boundaries, so I took population from WorldPop grids and added it up for each ward. Roads, land-use tags and building footprints came from OpenStreetMap. I first tried the Overpass API, but it was not reachable from my network, so I downloaded the Geofabrik southern-zone extract and read it directly. All layers were projected to UTM zone 43N (EPSG:32643) so that areas and distances are in metres. The ward file also had surveyor names and phone numbers, and I removed these because they are personal data.

Ward polygons that are drawn by hand can have geometry problems, such as boundaries that cross themselves or badly formed rings. These can make overlays and buffers fail or give wrong areas. So after reprojecting, I ran *make_valid()* in GeoPandas on all the ward polygons. After that I calculated the ward areas and checked that there were still exactly 101 wards with unique numbers. Only then did I merge the wards into one corporation outline. The merge left thin slivers in the small gaps between neighbouring wards, so I closed them with a 10 m buffer out and back in.

2.2 Satellite composites

Kerala is cloudy for most of the year, so I only used dry-season images (November to April) and took the median of all clear observations in Google Earth Engine. For 2000 and 2010 I had to combine Landsat 5 and 7 over two dry seasons, because Landsat 5 alone had no usable scenes for 2000. I used Landsat 8 for 2017, Landsat 8 and 9 for 2025, and Sentinel-2 with Cloud Score+ masking ($cs_cdf \geq 0.60$) for 2019 and 2025. Every composite covers at least 98.6% of the area, with 7 to 29 clear observations per pixel on average (median). I added NDVI, NDBI and MNDWI to the six reflectance bands.

2.3 Land-cover classification

I mapped five classes: built-up, tree cover, other vegetation, water and bare/sand. Getting good training labels was the hardest part of the whole project. My plan was to take labels from global land-cover products, but they disagree a lot in this city. Dynamic World calls about 80% of the corporation built-up, while ESA WorldCover calls most of it trees. Both are looking at the same coconut-covered homesteads and reading them differently. In the end I only took Sentinel-2 training points where WorldCover and Dynamic World agree, with an extra check against GHSL built-up density. For Landsat, I used my Sentinel-2 2025 map aggregated to 30 m as the labels, so that both resolutions use the same class definitions.

The classifier was a Random Forest with 200 trees ($\sqrt{p}$ features per split). For each image I used about 7,000 training points and kept about 3,000 for testing. The points were sampled in spatial blocks so that test points are not sitting right next to training points. I also wanted to use land surface temperature as a feature, but Landsat LST is missing over roughly 20% of the area in every year, so I left it out.

2.4 Making the years consistent

My first try was to classify each year on its own and compare the maps. This gave almost no growth in built-up land (20.9% to 22.9%), which clearly did not match GHSL (13.2% to 21.3%). The reason is that the small errors in each year's map add up and look like change. So I changed the approach. I started from the 2025 map and went back one epoch at a time. A pixel keeps its later class unless its spectral values changed by more than k standard deviations between the two composites; only then does it take that year's own class. I tested k from 0 to 1.5 and picked $k = 0.25$, because its built-up trend fitted GHSL best (squared error 0.011). At 30 m the model could not reliably tell tree cover from other vegetation, so in the results I add them together as total green.

2.5 Big data processing

I turned every pixel into a point carrying its class history: 451,000 points at 30 m and 4.04 million at 10 m. These were processed in two systems:

- **Apache Sedona on Spark** (local mode on my laptop). I joined pixels to wards in four ways: plain Spark with no spatial index (nested loop), a Sedona broadcast join with an R-tree, and Sedona partitioned joins with KDB-tree and Quadtree partitioning. I compared them with GeoPandas at 0.1, 0.5, 1 and 4 million points. I also used Sedona to find the distance from every green pixel to the nearest main road, and to sum the change into 2,082 H3 hexagons (resolution 9).
- **PostgreSQL 13 with PostGIS.** I ran the same joins, a 1 km range query around Technopark and a nearest-road (KNN) query, each with and without a GiST index. I also split the pixel table into 30 range partitions to test partition pruning, and saved the query plans with EXPLAIN ANALYZE.

To check the joins, I compared the ward totals from Sedona, PostGIS and the raster pipeline. They differed by at most 0.46 percentage points.

2.6 Analysis and decision support

- **Hotspots.** Global Moran's I and Getis-Ord Gi* on the hexagons (6 nearest neighbours, 999 permutations, false discovery rate correction), and DBSCAN (eps 100 m, 12 pixels) on the converted pixels to find compact building sites.
- **Association rules.** Every pixel that was green in 2000 became one transaction made of simple attributes: how built-up its surroundings were in 2000, distance to the centre, roads and Technopark, elevation, slope, low-lying or not, ward density, vegetation type and OSM land use. I ran FP-Growth in Spark MLlib (min. support 0.003, min. confidence 0.10) and kept the rules that end in “built-up” or “stayed green”.
- **Case studies.** Heat (MODIS at 1 km, and Landsat LST), flooding (low-lying land with MERIT Hydro HAND ≤ 5 m, and the 2018 flood mapped from Sentinel-1) and green area per person.
- **Ward priority index.** Seven ward indicators (long-term and recent green loss, share in hotspots, low-lying green converted, LST in 2025, green per person and density), min-max scaled and averaged. I tried three other weightings to see if the ranking holds.
- **Dashboard.** A Streamlit app that shows the maps, tables and ward rankings to people who are not GIS specialists.

3. Results

3.1 Classification accuracy

Table 1. Random Forest accuracy on the held-out test points.

Composite	Train / test	Overall acc.	Kappa	F1 built-up	F1 tree	F1 other veg.	F1 water
Sentinel-2 2019	6,857 / 3,143	0.959	0.948	0.926	0.966	0.924	1.000
Sentinel-2 2025	6,786 / 3,214	0.958	0.947	0.962	0.933	0.944	1.000
Landsat 2000	6,989 / 3,011	0.709	0.635	0.722	0.692	0.428	0.984
Landsat 2010	6,741 / 3,259	0.757	0.696	0.699	0.738	0.509	0.960
Landsat 2017	6,990 / 3,010	0.823	0.778	0.724	0.814	0.592	0.994
Landsat 2025	7,154 / 2,846	0.819	0.773	0.771	0.817	0.609	0.993

The test labels come from the same reference products as the training labels, so these figures show agreement with those products, not with field data.

The Sentinel-2 maps reached about 96% accuracy. Landsat is lower (71–82%) and gets worse for older images, mostly because other vegetation gets mixed up with tree cover (F1 only 0.43–0.61). That is why I

merged the two into one green class. Built-up and water were classified well in every year.

3.2 How much green was lost

Table 2. Green-cover change in the corporation (backdated Landsat series, km²).

Period	Green at start	Green at end	Green lost	of which to built-up	Green gained	Net change	Loss per year
2000–2010	165.15	161.50	7.10	5.26	3.44	−3.65 (−2.2%)	0.71
2010–2017	161.50	159.35	4.37	2.94	2.22	−2.15 (−1.3%)	0.62
2017–2025	159.35	157.72	4.19	3.43	2.56	−1.63 (−1.0%)	0.52
2000–2025	165.15	157.72	13.34	12.17	5.91	−7.43 (−4.5%)	0.53

Built-up land grew from 33.2 km² (15.6%) in 2000 to 48.8 km² (22.9%) in 2025, an increase of 47%. Green cover went down from 77.7% to 74.2%. Of the 13.3 km² of green that was lost, 12.2 km² (91%) became built-up and the rest became water or bare land. Part of the loss was balanced by 5.9 km² of new green, mostly on bare and sandy land along the north-west coast. The yearly loss got smaller in each period, from 0.71 to 0.52 km².

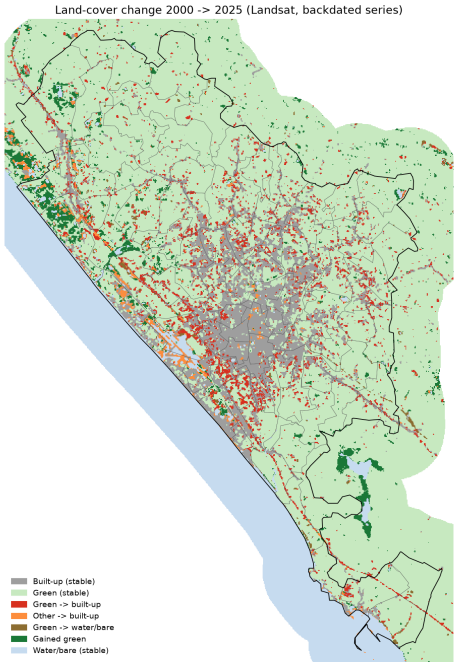


Figure 1. Land-cover change 2000–2025. Red is green land that became built-up. The conversion spreads out from the dense core (grey) along the coastal highway to the south and along the main roads to the north and east. Green gains (dark green) are mostly on the north-west coast.

3.3 Big data processing performance

Table 3. Point-in-polygon join of pixels to the 101 wards, run time in seconds.

Points	Spark nested loop	Sedona broadcast R-tree	Sedona Quadtree	Sedona KDB-tree	GeoPandas (STRtree)
0.10 M	4.4	0.31	0.67	1.13	0.04
0.50 M	19.6	0.37	1.93	3.03	0.31
1.00 M	40.2	0.71	3.83	6.56	0.76
4.04 M	192.0	3.00	17.56	34.00	1.63

Using a spatial index made the biggest difference. At 4 million points the Sedona broadcast R-tree join took 3 seconds, against 192 seconds for plain Spark, so it was about 64 times faster. Because there are only 101 ward polygons, sending the whole ward layer to every worker worked better than partitioning

both sides. Between the two partitioners, Quadtree gave more even partitions than KDB-tree (coefficient of variation of points per partition 0.44 against 1.79) and ran about twice as fast. Spark's default hash partitioning makes every partition the same size, but each partition covers 99.9% of the city's extent, so it gives no spatial pruning at all. What surprised me was that GeoPandas was still the fastest on my laptop. At this data size Sedona is not useful for raw speed on one machine; its value is that the same code can scale to a cluster and to a larger area.

Table 4. PostGIS queries with and without a GiST index (seconds).

Query	No index	GiST index	Speed-up
451 k pixels to wards (point-in-polygon)	4.29	0.53	8×
203 k building centroids to wards	20.60	0.87	24×
Range query: converted pixels within 1 km of Technopark	0.073	0.009	8×
KNN: nearest main road for sampled green pixels	14.54	0.13	114×
Partitioned table: 1 of 30 partitions vs all 30	0.076	0.010	8×

In PostGIS the GiST index replaced sequential scans with index scans. The KNN query gained the most (114×) because the <-> operator can walk the index in order of distance. With partitioning, PostGIS read only 1 of the 30 partitions when the query filtered on the partition key. PostGIS and Sedona both gave the same green to built-up total of 12.17 km².

3.4 Where the loss is concentrated

The conversion is clearly clustered (Moran's I = 0.485 for hexagons and 0.343 for wards, p = 0.001). Gi* found 262 hot hexagons at the 95% level (115 after FDR correction), holding 4.55 km² of conversion. DBSCAN grouped 57.5% of all converted pixels into 250 compact clusters. The largest are in Sreevaraham (0.44 km²), Kalippankulam (0.40 km²) and Chakkai (0.28 km²), all south of the core near the bypass and the airport. The cold spots, where green was kept, are along the northern, eastern and southern edges of the corporation.

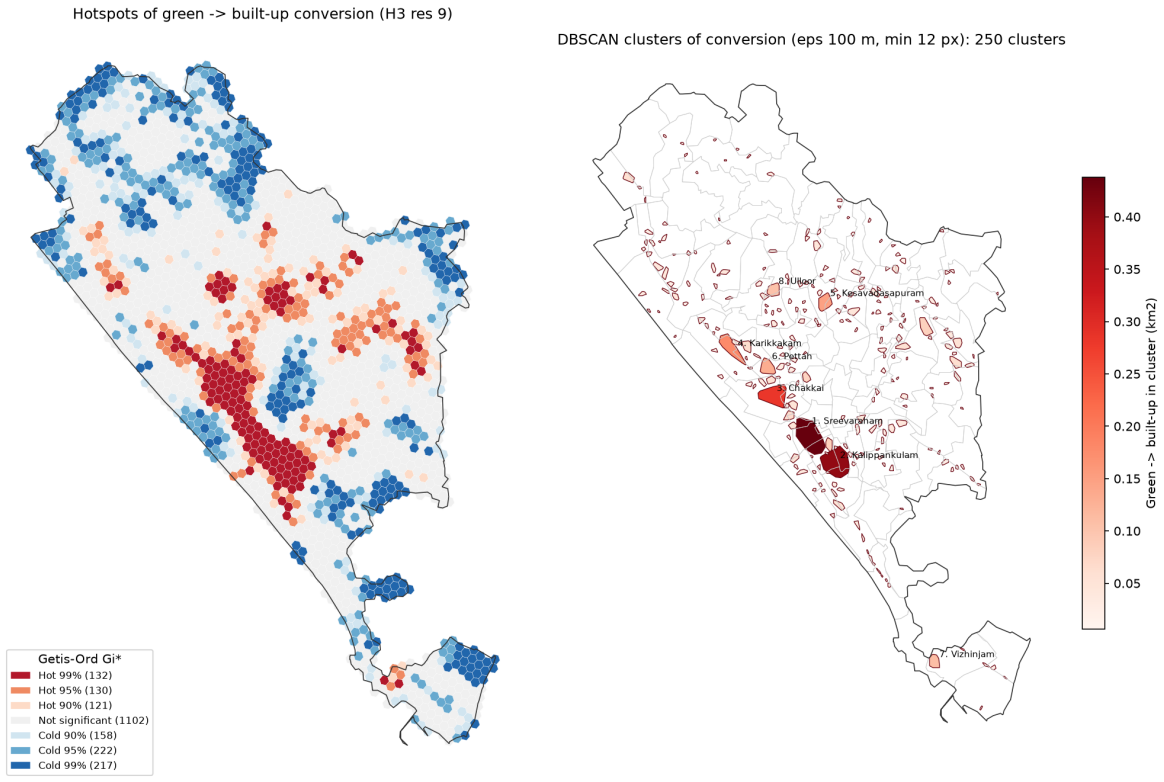


Figure 2. Left: Getis-Ord Gi* hot and cold spots of green to built-up conversion on H3 hexagons. Right: DBSCAN clusters of converted pixels, with the eight largest labelled by ward.

3.5 What makes conversion more likely

Roads matter a lot. Green land within 100 m of a road was converted at 12.4%, but only 2.3% was converted at 1–2 km from a road. Overall, 61% of all conversion happened within 250 m of a road.

Table 5. Strongest FP-Growth rules (182,192 transactions; the base conversion rate is 7.42%).

Condition (green pixel in 2000)	Outcome	Area (km²)	Confidence	Lift
high built-up around it, <3 km from centre, flat	built-up	1.35	39.4%	5.31
high built-up around it, elevation <10 m	built-up	1.67	35.4%	4.76
high built-up around it, flat, high ward density	built-up	2.21	35.2%	4.75
high built-up around it, elevation 10–30 m	built-up	7.78	29.8%	4.02
OSM land use = forest	stayed green	2.46	99.3%	1.07
OSM land use = agriculture	stayed green	2.69	99.3%	1.07
>10 km from centre	stayed green	34.51	97.1%	1.05

FP-Growth found 146 rules for conversion and 17 for staying green. The strongest pattern is infill. Green land that was already surrounded by built-up land in 2000 was converted at 27%, compared with 3.5% where the surroundings were green. Flat, low land near the centre was converted most often. Land tagged in OSM as forest, agriculture or wetland almost always stayed green.

3.6 Heat, flooding and green per person

- **Heat.** Greener wards are cooler. Ward green share against MODIS LST 2025 gives Spearman $\rho = -0.77$ ($p < 10^{-20}$). In 2025, Landsat LST averaged 38.7 °C on built-up land and 34.6 °C under tree cover. Pixels that changed from green to built-up warmed by 3.0 °C between 2000 and 2025, against 1.5 °C for pixels that stayed green, so about 1.6 °C extra.
- **Low-lying land.** 7.8% of low-lying green land (HAND ≤ 5 m) was converted (2.95 km²), a little more than on higher ground (7.2%). The 2018 flood extent I mapped inside the corporation was only 0.32 km², with almost no conversion inside it, so I discuss flood risk through low-lying land instead.
- **Green per person.** Green area per resident fell from 180 m² in 2000 to 171 m² in 2010 (WorldPop 2000–2020 release), and from 139 m² in 2017 to 132 m² in 2025 (WorldPop R2025A). The number of wards below 50 m² per person went up from 17 to 27. The two WorldPop releases cannot be compared with each other, so I did not interpret the 2010 to 2017 step.

3.7 Ward priority

The top five wards on the priority index are Sreevaraham (95/100), Perunthanni (88), Kamaleswaram (86), Pettah (77) and Kalippankulam (74), all in the southern part of the core. The ranking is stable. Across the four weightings, the Spearman correlation with the equal-weight ranking is at least 0.967, and 16 to 19 of the top 20 wards stay in the top 20.

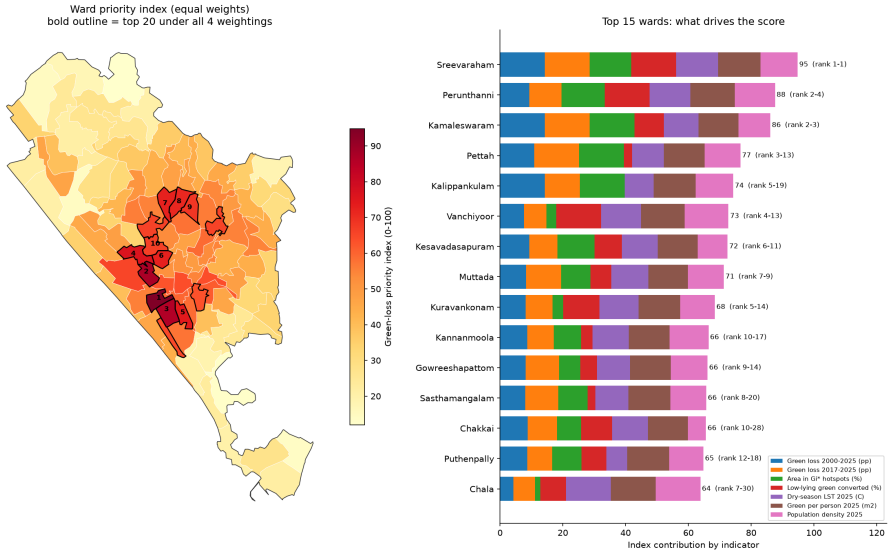


Figure 3. Ward priority index (equal weights). Left: map, with a bold outline for wards that are in the top 20 under all four weightings. Right: what each indicator adds for the 15 highest-scoring wards, with the range of ranks across weightings.

4. Discussion

4.1 What the results mean

Thiruvananthapuram lost about 4.5% of its green cover in 25 years. That sounds small next to the 47% growth in built-up land, but the city started out very green and a lot of the new building happened inside existing homesteads. The loss is also not spread evenly. It is concentrated in a belt south and west of the old city (Sreevaraham, Kalippankulam, Kamaleswaram, Chakkai, Pettah) and along the highways, where some wards lost 18 to 30 percentage points of their green cover. The rules and the road-distance results point to the same process: the city fills in gaps next to existing buildings and roads instead of jumping out into open land. The slower loss after 2010 could be because little green is left near the core, or because of something else; my data cannot separate the two.

The heat results show why this matters in practice. Land that was built on warmed 1.6 °C more than land that stayed green, and the greenest wards are the coolest. The priority wards are also among the hottest and most crowded, so protecting the green that is left there would help with heat directly. Land recorded as agriculture, forest or wetland in OSM was almost never built on, which suggests that formal land-use designation does protect green land.

4.2 What I learned about the big data tools

Three ideas from the course showed up clearly in my results. First, a spatial index matters more than the choice of engine. An R-tree in Sedona or a GiST index in PostGIS gave speed-ups of 8 to 114 times, while Spark without spatial support was slow. Second, partitioning has to follow the data. Hash partitioning balances the load but ignores location, while Quadtree partitions adapted to the dense core. Third, for one city with 4 million points a single machine is still enough. Sedona would become necessary if I extended the work to the whole state, to 10 m pixels for every year, or to building-level analysis. PostGIS was the easiest for repeated ward and distance queries, and Spark was the natural place to run FP-Growth on 182,000 transactions. Running the same joins in both systems was also a good way to check that my numbers were right.

4.3 Limitations

- Accuracy was checked against labels from global products, not field data. Landsat 2000 has the lowest accuracy (71%) and its 6.7% bare/sand share looks too high. This causes the apparent green gains in north-western coastal wards such as Pallithura.

- At 30 m, a homestead with a new house under coconut trees often still looks green, so conversion is probably underestimated, especially before 2017.
- The backdated series depends on k and on GHSL. With k from 0 to 0.5, the total green loss ranges from 4.3 to 2.7 percentage points (3.5 at k = 0.25).
- MODIS LST is coarse (1 km), Landsat LST has gaps, the two WorldPop releases cannot be compared across 2010–2017, and OSM is not equally complete everywhere.
- Sedona only ran in local mode on one machine. I did not test a real multi-node cluster.

4.4 Conclusion and future work

Between 2000 and 2025, Thiruvananthapuram lost 12.2 km² of green land to building, mostly through infill near roads and existing built-up areas in the southern and central wards. These same wards are the hottest and have the least green per person. The priority index stays stable under different weightings, so it gives the corporation a reasonable shortlist of wards for protecting green space and planting trees. If I continue this work, I would add field validation points, use building footprints from different years to catch houses hidden under trees, run the workflow on a real Spark cluster for the surrounding panchayats, and update the dashboard every dry season.

References

1. Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5–32.

2. Brown, C. F. et al. (2022). Dynamic World, near real-time global 10 m land use land cover mapping. *Scientific Data*, 9, 251.

3. Zanaga, D. et al. (2022). ESA WorldCover 10 m 2021 v200. Zenodo.

4. Pesaresi, M. et al. (2023). GHS-BUILT-S R2023A – GHS built-up surface grid. European Commission, Joint Research Centre.

5. Gorelick, N. et al. (2017). Google Earth Engine: planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, 202, 18–27.

6. Yu, J., Zhang, Z., Sarwat, M. (2019). Spatial data management in Apache Spark: the GeoSpark perspective and beyond. *GeoInformatica*, 23, 37–78.

7. Han, J., Pei, J., Yin, Y. (2000). Mining frequent patterns without candidate generation. *Proc. ACM SIGMOD*, 1–12.

8. Getis, A., Ord, J. K. (1992). The analysis of spatial association by use of distance statistics. *Geographical Analysis*, 24(3), 189–206.

9. Ester, M., Kriegel, H.-P., Sander, J., Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proc. KDD*, 226–231.

10. Yamazaki, D. et al. (2019). MERIT Hydro: a high-resolution global hydrography map. *Water Resources Research*, 55, 5053–5073.

11. WorldPop (2018, 2025). Global high-resolution population denominators. University of Southampton.

12. Kerala Local Self Government Department (2024). K-SMART ward delimitation boundaries.